

Data Handling: Import, Cleaning and Visualisation

Example Solution for Exercise/Workshop 2: Computer Code and Data Storage

Aurélien Sallin (lecturer) and Matthias Roesti (class tutor)

06/10/2023

Exercises - example solution

Exercise A: Loading text files in RStudio

When working with data, a first important step is to load your data, in our case in RStudio. As you know, data can come in different formats.

1. Along with the exercise instructions, you will find two datasets on Canvas, `simple_data.txt` and `simple_data.csv`. Download them and save/upload them into the `r_course/data` folder (create data subfolder if not already existing). Further ensure that your working directory is the `r_course` folder.
2. Check out the files `simple_data.txt` and `simple_data.csv` by opening them (outside of RStudio) in a text editor, such as Sublime Text or Atom. Do they look different from each other?

We can see that the files contain the same information, but do not look the same. The `simple_data.csv`-file contains values separated by a comma (,), while in the `simple_data.txt`-file we have data columns that are separated by a tabstop.

3. Go back to RStudio and load the data in .txt format as follows:

```
df1 <- read.table("data/simple_data.txt", header = FALSE)
```

Type `df1` or `print(df1)` into the console. What do you see?

We see the dataframe printed out in full with 2 observations for 3 variables (note that here `df1` and `print(df1)` produce the same result). We further see that R has automatically assigned variable names `V1`, `V2` and `V3` because the file did not provide any variable names.

4. Next, turn to the file in .csv format. Run the following code to load it:

```
df2 <- read.table("data/simple_data.csv", header = FALSE, sep=",")
```

Again, type `df2` or `print(df2)` into the console. Now, what do you see? Compare this output to the output from `print(df1)`.

We can see that the dataframes are identical, despite them looking different in the raw format. This is important for our analysis, because to efficiently work with data further down our pipeline, we need to ensure that we can bring it into a format that the software and its functions can reliably work with. In the present case, both input files get converted into a `data.frame` object in R - something which you can also verify in the documentation for the reading function we used (`?read.table` will tell you that the function ‘Reads a file in table format and creates a data frame from it’).

You can also check that the dataframes are identical objects (sidenote: this function tests for exact equality of objects, which is even more strict than just comparing whether values match)

```
identical(df1,df2)
```

```
## [1] TRUE
```

5. Generate a text file containing the value in the third column of the second row of `df1`. Hint: you can obtain values from dataframes by indexing like this: `df[i,j]`, where `i` and `j` indicate row and column numbers, respectively. Save the text file as “`df1_value.txt`” in your `r_course/data` folder.

```
cat(df1[2,3], file = "data/exercise2_df1_value.txt")
```

The text file should contain a b

Exercise B: Downloading and importing data on air quality

In the following, we are going to download and import data on air quality in New York.

1. Using a browser of your choice, navigate to this page by the US data.gov data catalog: <https://catalog.data.gov/dataset/air-quality>.
2. Scroll down to the Section ‘Downloads & Resources’ and download the data file in .csv format and put it into your `r_course/data` folder.
3. Based on Exercise A, Part 4, how would you open this dataset in RStudio? Hint: In contrast to the files in Exercise A, this file comes with a header row. Complete the following code accordingly:

```
df_airquality <- read.table(...)
```

The completed command reads

```
df_airquality <- read.table("data/Air_Quality.csv", header = TRUE)
```

4. Once you have loaded the data into RStudio, you can inspect the first (or last) couple of rows using `head(df_airquality)` (or `tail(df_airquality)`).

```
head(df_airquality)
```

##	Unique.ID	Indicator.ID	Name	Measure	Measure.Info	Geo.Type.Name	Geo.Join.ID
## 1	216498	386	Ozone (O3)	Mean	ppb	CD	313
## 2	216499	386	Ozone (O3)	Mean	ppb	CD	313
## 3	219969	386	Ozone (O3)	Mean	ppb	Borough	1
## 4	219970	386	Ozone (O3)	Mean	ppb	Borough	1
## 5	164876	383	Sulfur Dioxide (SO2)	Mean	ppb	CD	211
## 6	164877	383	Sulfur Dioxide (SO2)	Mean	ppb	CD	212
##	Geo.Place.Name		Time.Period	Start_Date	Data.Value	Message	
## 1	Coney Island (CD13)		Summer 2013	06/01/2013	34.64	NA	
## 2	Coney Island (CD13)		Summer 2014	06/01/2014	33.22	NA	
## 3	Bronx		Summer 2013	06/01/2013	31.25	NA	
## 4	Bronx		Summer 2014	06/01/2014	31.15	NA	
## 5	Morris Park and Bronxdale (CD11)		Winter 2008-09	12/01/2008	5.89	NA	
## 6	Williamsbridge and Baychester (CD12)		Winter 2008-09	12/01/2008	5.75	NA	

5. Create a text file with the amount (in variable `Data.Value`) for fine particulate matter pollution (variable `Name == "Fine Particulate Matter (PM2.5)"`) in the winter of 2008/09 (variable `Time.Period == "Winter 2008-09"`) in lower Manhattan (variable `Geo.Place.Name == "Lower Manhattan"`). Save your text file as “`airquality_manhattan.txt`” in your `r_course/data` folder. Hint: use the `subset()` command, and note that you can link different subsetting conditions with the `&` operator.

Here, we first need to find the location of the information we are looking for. One option to do this is the built-in `subset()` function, using the `&` logical operator to connect the different conditions needed. The `==`

operator is a comparison operator testing for logical equality, returning *TRUE* if the condition holds, and *FALSE* otherwise. Using just one `=` would produce an error because *R* expects a logical statement here.

```
my_info <- subset(df_airquality,
  Name == "Fine Particulate Matter (PM2.5)" &
  Geo.Place.Name == "Lower Manhattan" &
  Time.Period == "Winter 2008-09"
)

my_value <- my_info$Data.Value

cat(my_value, file = "data/airquality_manhattan.txt")
```

The resulting file should contain the value 15.79

Exercise C: Exploring computer code - HTML - let's manipulate the media!

In this exercise, we will explore HTML code, which is the basis of most websites.

1. Open your preferred web browser (e.g. Firefox, Chrome or Safari. Note that Internet Explorer is never a preferred browser). Go to the following website: <https://www.foxnews.com/>.
2. Now let's look at the HTML source code for this website - see *here* for information on how to do this in your browser of choice. Try to recognize parts of the code that correspond to elements of the displayed page.
3. Pick a news headline on the site and find it in the source code - on Windows, you can use **CTRL + F** to use the search bar to look for it in the source code by copying the first couple of words from the headline and putting it into the search bar. Note: most browsers also have an 'inspect element' option when you right click on/select the element you want to look at. This can take you directly to the corresponding part of the HTML (or at least to the vicinity, sometimes you also have to try a couple of times to get where you want to be). For more information on how to do this (in Safari, you will need to activate developer mode for example), see the following *link*

For example, I found the following segment in the HTML code relating to an article titled 'Republicans sound alarm over CCP-linked businessman's cash offer to Hunter Biden' (no subtle political messaging intended):

```
<!DOCTYPE html>
<html>

<head>
...
</head>

<body>
...

<article class="article story-2">
  <div class="m">
    <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy-
relationship-ccp-linked-businessman-damning-evidence" data-omtr-intcmp="fnhpb3" data-adv=
"hp1bt">

      <picture>
        <source media="(max-width: 767px)" srcset="//a57.foxnews.com/prod-hp.fox
```

```

news.com/images/2023/10/343/193/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&ve=1, //a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/686/386/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&ve=1 2x">
    <source media="(min-width: 768px) and (max-width: 1023px)" srcset="//a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/335/188/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/670/376/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&ve=1 2x">
    <source media="(min-width: 1024px) and (max-width: 1279px)" srcset="//a57.
foxnews.com/prod-hp.fox
news.com/images/2023/10/303/170/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/606/340/57
6c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&ve=1 2x">
    <source media="(min-width: 1280px) and (max-width: 1439px)" srcset="//a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/277/156/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&ve=1, //a57.foxnews.com/prod-hp.foxnews.com/images/2023/10/554/312/576c017a6f
05588ac891d4aa76dbd0e7.jpg?tl=1&ve=1 2x">
    <source media="(min-width: 1440px)" srcset="//a57.foxnews.com/prod-hp.fox
news.com/images/2023/10/331/186/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&ve=1, //a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/662/372/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&ve=1 2x">
    
    </picture>
    <div class="kicker default"><span class="kicker-text">'IN THEIR POCKET'</span>
    </div>
    </a>
    <span class="pill is-breaking">EXCLUSIVE</span>
</div>

<div class="info ">
    <header class="info-header">
        <h3 class="title ">
            <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy
-relationship-ccp-linked-businessman-damning-evidence#&_intcmp=fnhpbt3"
data-omtr-intcmp="fnhpbt3">Republicans sound alarm over CCP-linked business-
man's cash offer to Hunter Biden</a>
        </h3>
    </header>
    <div class="content">

    </div>
</div>

</article>

...

</body>

</html>

```

4. Now it's time for some good old media manipulation. In the HTML-code, replace the chosen news headline with a title of your choice (if you need some inspiration, see *here*). In most browsers, you can double click the source code to edit it, then confirm your edits by hitting the enter/return key. What can you see? Did the website change?

For example, we could change the displayed story title by changing the text of the element:

```
<!DOCTYPE html>
<html>

<head>
...

</head>

<body>

...

    <div class="info ">
        <header class="info-header">
            <h3 class="title ">
                <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy-relationship-ccp-linked-businessman-damning-evidence#&_intcmp=fnhpbt3" data-omtr-intcmp="fnhpbt3">In surprise move, Republicans turn to full support for Joe Biden. Country now in peace and harmony.</a>
            </h3>
        </header>
    </div>

</article>

...

</body>

</html>
```

5. BONUS (not exam relevant): Can you figure out a way to replace the article teaser image as well? Hint: this is easiest if you use online image addresses, e.g. https://static.wikia.nocookie.net/mrbean/images/4/4b/Mr_beans_holiday_ver2.jpg or <https://preview.redd.it/bad-luck-brian-meme-hd-upscaled-v0-syiviojoq72a1.jpg?width=1080&crop=smart&auto=webp&s=eace4bc6be838bfe3f30219743d5a5c34c5f4335>)

The simplest way of doing this is to replace the link to the image with a link to another image in the element (note that I also removed all the sub-elements starting with `<source media=?`, these are just what the webpage authors implemented to allow for choosing different image files depending on the screen size - you would have to replace each image link to ensure full replacement in case you were to keep those bit in)

```
<!DOCTYPE html>
<html>

<head>
...
```

```

</head>

<body>

...

  <picture>
    
  </picture>

...

</body>

</html>

```

As a sidenote, the `alt="In surprise move, Republicans turn to full support for Joe Biden. Country now in peace and harmony."` part is not the same as the title we changed earlier, it defines the alternative text that describes the image, which serves purposes like assisting accessibility and search engine optimization (SEO).

Exercise D: Importing data on air quality (XML format)

1. I provided you with a data file `Air_Quality.xml` on Canvas. Save it in your `r_course/data` folder.
2. Load the data in .xml format. Hint: The package `XML` might be of use. You can install new packages as follows: `install.packages("package_name")`.

```

# installing package if not already installed - the require function here will output TRUE
# if XML is already installed - the `!` (read 'not') in front of it means that we only
# want to install it if this is not the case
if (!require(XML)) {
  install.packages("XML")
}

library(XML) # loading the package after install - need to run every time we use it!

# Load the data from the XML file
xml_data <- xmlParse("data/Air_Quality.xml")

# Convert XML data to a data frame
air_quality_data <- xmlToDataFrame(xml_data)

str(xml_data) # Checking the structure of our xml_data

```

We can see that we loaded an object, but instead of being a nice neat dataframe, what we got was actually an R structure representing the XML tree, i.e. a structure that contains all the information we need to extract the data we want, but we cannot use this directly like a data frame

3. Once you have loaded the .xml file in Part 2, how can you convert it to an R dataframe?

Luckily, the `XML` package offers a convenient function to convert our parsed structure into a standard data frame:

```
# Convert XML data to a data frame
```

```
air_quality_data <- xmlToDataFrame(xml_data)
```

```
str(air_quality_data) # Checking structure again
```

```
## 'data.frame':    16122 obs. of  11 variables:
## $ unique_id      : chr  "216498" "216499" "219969" "219970" ...
## $ indicator_id   : chr  "386" "386" "386" "386" ...
## $ name           : chr  "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" "Ozone (O3)" ...
## $ measure        : chr  "Mean" "Mean" "Mean" "Mean" ...
## $ measure_info   : chr  "ppb" "ppb" "ppb" "ppb" ...
## $ geo_type_name  : chr  "CD" "CD" "Borough" "Borough" ...
## $ geo_join_id    : chr  "313" "313" "1" "1" ...
## $ geo_place_name : chr  "Coney Island (CD13)" "Coney Island (CD13)" "Bronx" "Bronx" ...
## $ time_period    : chr  "Summer 2013" "Summer 2014" "Summer 2013" "Summer 2014" ...
## $ start_date     : chr  "2013-06-01T00:00:00" "2014-06-01T00:00:00" "2013-06-01T00:00:00" "2014-06-01T00:00:00" ...
## $ data_value     : chr  "34.64" "33.22" "31.25" "31.15" ...
```

We have a data frame now - with the caveat that the reading function defaulted to declaring all variables as strings. In order to be able to work with numerical columns, we would first have to convert them into numeric variables, which can be done like this for example: `air_quality_data$data_value <- as.numeric(air_quality_data$data_value)`. For our purposes, importing data from a .csv or text file is often easier than parsing XML files (note also the file size of the XML data relative to the csv file, in itself not the most compact format either).

Exercise E: Read raw text data

This exercise guides you through how we can import raw text data into R. Suppose you are writing your BA Thesis on the topic “The Effect of Climate Change on Consumption Behavior in Switzerland”. For your empirical analysis you need, inter alia, standardized data on temperature anomalies in Switzerland (as a proxy for the salience of climate change). You identify NASA’s Earth System Data Explorer as a promising data source and aim to download and work with this data in R. The exercise guide’s you exemplary though how this can be done.

Hint: This exercise is an adapted version of the tutorial in Chapter 1 of Paul Murell’s book (see syllabus). Read the chapter for an additional introduction to data stored in text files.

1. Go to NASA’s Earth System Data Explorer: <https://myasadata.larc.nasa.gov/EarthSystemLAS/UI.v>m.
2. Click on *Data Set* in the upper left corner and navigate through *Atmosphere/Featured Phenomena/Changing Air Temperatures/Temperatures* and select *Monthly Surface Air Temperature Anomaly*.
3. On the bottom left part of the page, under *Line Plots*, select *Time*.
4. Enter the coordinates 47.25 N and 9.22 W (St. Gallen) and click *Update Plot* in the upper left corner. The page shows a plot of the monthly surface air temperature anomaly time series.
5. Click on *Download Data* In the new tab select *ASCII* as a data format. Click to save.

The downloaded ASCII-file containing the raw data should open in a new tab. Inspect the data and save (Save page as...) it as `temp_anomaly.txt` in your `r_course/data` folder.

Now we can read the data into R:

```
# install the readr package if not yet installed: install.packages("readr")
library(readr)
```

```
# read the data
ta <- read_tsv("data/temp_anomaly.txt", skip = 9, col_names = c("month", "temp_anomaly"))

# inspect the first rows
head(ta)
```

```
## # A tibble: 6 x 2
##   month      temp_anomaly
##   <chr>          <dbl>
## 1 JAN-1880        0.33
## 2 FEB-1880        0.3
## 3 MAR-1880        0.47
## 4 APR-1880        0.15
## 5 MAY-1880       -0.36
## 6 JUN-1880       -0.43
```

- What is the purpose of the `skip` argument and why is it set to 9?

In the raw text file, we see that the first 9 lines contains metadata and are in a different format - therefore we need to skip these lines when reading the data into R. Also note how we specified the column names in our command to read in the data, because the data did not have the variable names included in the columns.

- Why do we use `read_tsv()` in this situation and not `read_csv()`?

The raw data shows us that columns are not separated by a comma, but rather by a tab, hence we have what is called a tab-separated values (TSV) file. That said, we could also use the more general `read_delim()` function, specifying the tab `\t` as a separator:

```
ta <- read_delim("data/temp_anomaly.txt", skip = 9, delim = "\t",
                col_names = c("month", "temp_anomaly"))
```